DEPARTMENT OF INDUSTRIAL MANAGEMENT

FACULTY OF SCIENCE | UNIVERSITY OF KELANIYA

INTE 22283 – Mobile Application Development

Week 3: Mobile Development Platforms

Sammani Maddumahewage

Lecturer (Probationary)

BSc (Hons) in Information Technology



Today's Learning Outcomes

- Differentiate between native, hybrid, and cross-platform development approaches
- Evaluate the advantages and limitations of each development approach
- Identify popular native development tools (Android Studio, Xcode)
- Introduce cross-platform frameworks (Flutter, React Native)
- Compare performance, cost, time-to-market, and maintainability across platforms
- Apply a decision framework for selecting a platform based on project requirements
- Configure a development environment for a chosen platform
- Build and run a simple “Hello World” application

Recap and Framing

- **Last two weeks:** We covered mobile networks and mobile operating system architecture — the foundation every app runs on
- **This week:** We address the practical question every project starts with: which development approach should we actually use to build the app?
- **Why it's not obvious:** There is no single “best” approach — the right choice depends on performance needs, budget, timeline, and team skills

Three Approaches to Building Mobile Apps

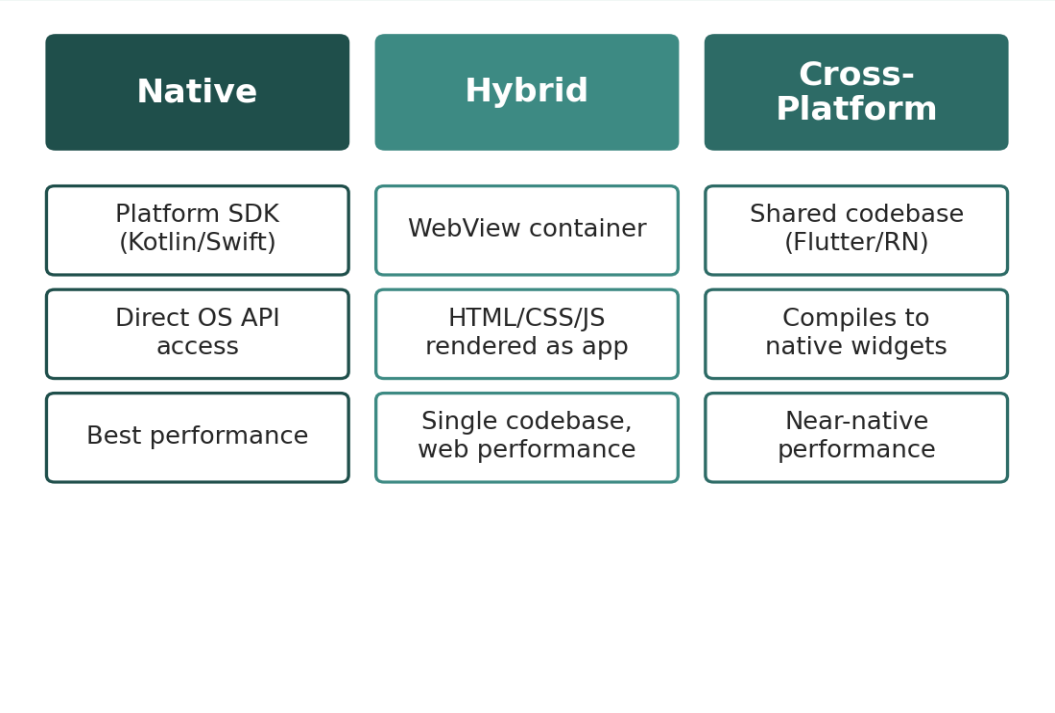


Figure 1: Native, hybrid, and cross-platform approaches differ in how code is written and executed on the device

- **Native:** Built using each platform's own language and SDK (Kotlin/Java for Android, Swift for iOS)
- **Hybrid:** A web app (HTML/CSS/JS) wrapped in a native container and rendered through a WebView
- **Cross-platform:** A single codebase compiled or rendered into native UI components on each platform

Native Development

- **Definition**
  - Apps built specifically for one platform using that platform's official language, SDK, and tools
- **Advantages**
  - Best possible performance; full access to all platform APIs and hardware features on day one
- **Limitations**
  - Requires separate codebases (and often separate teams) for Android and iOS, increasing cost and time
- **Typical use case**
  - Apps with demanding performance needs: games, AR/VR, or apps requiring deep hardware integration

Hybrid Development

- **Definition**
  - A web application (built with standard web technologies) packaged inside a native shell using a WebView
- **Advantages**
  - Single codebase across platforms; reuses existing web development skills and code
- **Limitations**
  - Performance and UI feel are constrained by the WebView; limited access to some native features without plugins
- **Typical use case**
  - Content-driven apps, internal business tools, or MVPs where speed of development matters more than peak performance

Cross-Platform Development

- **Definition**
  - A single codebase, written in one framework, that compiles to (or renders as) native UI components on each platform
- **Advantages**
  - Near-native performance and look-and-feel, while sharing most business logic and UI code across platforms
- **Limitations**
  - Occasional need for platform-specific code for cutting-edge native features; an additional abstraction layer to learn
- **Typical use case**
  - Most commercial apps today — a strong balance of cost, speed, and quality (e.g., built with Flutter or React Native)

Native Development Tools

Android Studio (for Android)

- **Built on:** JetBrains IntelliJ IDEA platform
- **Languages:** Kotlin (preferred) and Java
- **Includes:** Layout editor, emulator, profiler, Gradle build system

Xcode (for iOS)

- **Built by:** Apple, exclusively for macOS
- **Languages:** Swift (preferred) and Objective-C
- **Includes:** Interface Builder, Simulator, Instruments profiler

Cross-Platform Frameworks: Flutter and React Native

- **Flutter (Google):** Uses the Dart language; renders its own UI widgets directly to the screen rather than using native OS components, giving highly consistent visuals across platforms
- **React Native (Meta):** Uses JavaScript/TypeScript with React; renders using actual native UI components, giving a more platform-authentic feel
- **Shared principle:** Both let developers write most application logic once and deploy to Android and iOS, with optional platform-specific code where needed
- **We return to these in depth:** Week 12 covers hybrid and cross-platform frameworks hands-on, including setting up a real Flutter or React Native project

Comparing the Three Approaches

Table 1: No single approach wins on every dimension — the right choice depends on project priorities

Dimension	Native	Hybrid	Cross-Platform
Performance	Best	Lower	Near-native
Development cost	Highest (2x teams)	Lowest	Moderate
Time-to-market	Slowest	Fastest	Fast
Maintainability	Two codebases	One codebase	One codebase
Access to new OS features	Immediate	Delayed	Often delayed

Reading the Comparison Correctly

- **No universal winner:** Every approach makes a deliberate trade-off between performance, cost, speed, and long-term maintainability
- **Performance vs. cost tension:** Native maximizes performance and platform access at the highest cost; hybrid minimizes cost at some performance expense
- **Cross-platform as a middle ground:** Frameworks like Flutter and React Native increasingly close the performance gap with native, which is why they dominate new app development today
- **Maintainability compounds over time:** A single shared codebase becomes more valuable the longer an app is actively developed and maintained

A Decision Framework for Platform Selection

- **1. What does the app need to do?**
  - Heavy graphics, AR, or deep hardware access push toward native
- **2. What is the budget and timeline?**
  - Tight budgets and fast deadlines favor hybrid or cross-platform
- **3. What skills does the team already have?**
  - Web developers may ship faster with hybrid; JS/React teams may prefer React Native
- **4. How long will the app be maintained?**
  - Long-lived apps benefit more from a single, maintainable cross-platform codebase
- **5. Does the app need day-one access to the newest OS features?**
  - If yes, native is safest; cross-platform frameworks sometimes lag behind new OS releases

Worked Example: A Mobile Game Studio

- **Requirements:** High frame-rate 3D graphics, deep access to GPU and sensor APIs
- **Applying the framework:** Performance need is extremely high (favors native); budget allows separate platform teams
- **Decision:** Native development — or a specialized game engine (Unity/Unreal) that compiles natively — is the right choice here

Worked Example: A Startup's MVP

- **Requirements:** Launch on both Android and iOS within 8 weeks, with a 2-person development team and a limited budget
- **Applying the framework:** Time-to-market and cost are the dominant constraints; the app's needs are largely standard UI and API calls, not specialized hardware access
- **Decision:** Cross-platform (Flutter or React Native) is the strongest fit — one codebase, fast iteration, near-native quality

Worked Example: An Internal Company Tool

- **Requirements:** A simple internal app for employees to submit expense reports; performance is not a major concern
- **Applying the framework:** Low performance requirements, small user base, and a team that already builds the company's web portal
- **Decision:** Hybrid development lets the team reuse existing web code and skills, minimizing cost for a low-stakes internal tool

Configuring a Development Environment

- **Step 1:** Choose your platform approach based on the decision framework (this module focuses primarily on native Android with Android Studio)
- **Step 2:** Install the IDE (Android Studio) and accept required SDK licenses
- **Step 3:** Configure an Android Virtual Device (AVD) or connect a physical device with USB debugging enabled
- **Step 4:** Verify the environment by creating a new project from a template

Building Your First App: “Hello World”

- **Create a new project:** Use Android Studio's “Empty Activity” template, which generates a minimal working app
- **Examine MainActivity:** This file contains the onCreate() method where the app's first screen is set up
- **Examine activity_main.xml:** This layout file defines the UI — by default, a single TextView displaying “Hello World!”
- **Run the app:** Build and deploy to the emulator or a connected device to see the running app for the first time
- **What this demonstrates:** Even a minimal app already uses the Activity lifecycle, layout system, and build process introduced in previous weeks

CONCLUSION

Key Takeaways

- Native, hybrid, and cross-platform are three distinct approaches with different performance, cost, and maintainability trade-offs
- Native development (Android Studio, Xcode) offers the best performance and platform access but the highest cost
- Cross-platform frameworks like Flutter and React Native offer a strong balance and now dominate new app development
- Platform selection should follow a deliberate decision framework based on requirements, budget, timeline, and team skills
- A working development environment and a successfully built “Hello World” app are the practical foundation for everything that follows

Summary

- This week answered a foundational practical question: how should a team actually build a mobile app? We examined native, hybrid, and cross-platform approaches, comparing their performance, cost, time-to-market, and maintainability trade-offs.
- We applied a decision framework to real-world scenarios — a game studio, a startup MVP, and an internal tool — each landing on a different, well-justified platform choice.
- With a working development environment and a first running app in place, next week we move from platform choice to internal application architecture.

Discussion Questions

- If you were building a banking app, which approach would you choose and why? Which decision-framework factors matter most for this case?
- Why do you think cross-platform frameworks have grown so much in popularity over the last decade, relative to native and hybrid approaches?
- Can you think of a scenario where starting with hybrid development and later migrating to native (or cross-platform) might make sense as a business strategy?
- What risks does a team take on by choosing a cross-platform framework, given that new OS features sometimes arrive on native platforms first?

Thank You !

Questions?

Email: Sammani@kln.ac.lk | Department of Industrial Management